DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSE 489: Mobile Application Development
Lab Exam

Smart Geo-Tagged Landmarks

📌 Objective

Design and develop an **Android mobile application** that interacts with a **faculty-provided REST API** to manage and visualize **Smart Geo-Tagged Landmarks**.

Each landmark includes:

- Title
- Geographic location (latitude & longitude)
- Image
- Activity-based score (computed by server)

Your application must support **viewing, visiting, filtering, and offline handling** of landmarks.

⚠️ Important Rules

- You are **NOT allowed to build or modify the backend**
- You must use the **provided API only**
- Each student will receive a **unique API key**
- API behavior (e.g., scoring) may vary per student

🔗 API Details

You will be provided with:

`https://labs.anontech.info/cse489/exm3/api.php`

You must include your **ID** in every request as **key**:

`?action=get_landmarks&key=YOUR_KEY`

`View submission through online: https://labs.anontech.info/cse489/exm3`

📡 Available API Endpoints

1. Get Landmarks

GET

`?action=get_landmarks&key=YOUR_KEY`

Returns a list of landmarks with:

- id
- title
- lat
- lon
- image
- score
- visit_count
- avg_distance

---

2. Visit Landmark

POST

```
?action=visit_landmark&key=YOUR_KEY
```

Body (JSON):

```
{
  "landmark_id": 1,
  "user_lat": 23.7,
  "user_lon": 90.4
}
```

---

3. Create Landmark

POST

```
?action=create_landmark&key=YOUR_KEY
```

---

4. Delete Landmark (Soft Delete)

POST

```
?action=delete_landmark&key=YOUR_KEY
```

---

5. Restore Landmark

POST

```
?action=restore_landmark&key=YOUR_KEY
```

🧠 Core Functional Requirements

---

1. Landmarks Display

- Fetch landmarks from API
- Display:
  - Title
  - Image
  - Score
- Handle dynamic data correctly

---

2. Map View

- Show all landmarks on map
- Center map on Bangladesh
- Marker must:
  - Represent a landmark
  - Show details on click
- Marker color should reflect score (low → high)

---

3. Visit Feature (IMPORTANT)

- User can visit a landmark
- App must:
  - Get current GPS location
  - Send visit request
  - Display returned distance
- Show success/failure message

---

4. Landmarks List

- Show all landmarks in list form
- Each item must show:
  - Title
  - Score
  - Image
- Must support:
  - Sorting by score
  - Filtering by minimum score

---

5. Activity Screen (Visit History)

- Display recent visits
- Each entry must show:
  - Landmark name
  - Visit time
  - Distance

---

6. Add Landmark

- Input:
  - Title
  - Latitude & Longitude
  - Image
- Auto-fetch GPS location for new entry

Most Common mistakes:
If you use:

- `raw JSON` → ❌ `$_FILES` will be empty

For file upload you MUST use:

```
Body → form-data
```

---

7. Soft Delete Handling

- Deleted landmarks should not appear in list
- App must not crash if data changes

---

8. Offline Support (MANDATORY)

Your app must:

- Cache fetched data locally
- Display data when offline
- Queue visit requests when offline
- Sync queued requests when internet is available

---

9. Error Handling

- Show success messages using Toast/Snackbar
- Show errors using dialogs or messages
- Handle API failures gracefully

📱 UI Requirements

Use **Bottom Navigation** with 4 tabs:

1. Map
2. Landmarks
3. Activity
4. Add/View

📦 Submission Instructions

🔗 GitHub Repository

- Create a **private repository**
- Add instructor as collaborator (give permission to github handler: rahman9909)
- Submit repository link via form

📁 Required Structure

```
project-root/
│
├── app/ (contains your application codebase without Build folder)
│
├── Ai_usage.txt (your justification/spec of ai usage)
└── README.txt
```

📝 README.txt Must Include

1. Project Overview
2. Features Implemented
3. API Usage
4. Offline Strategy
5. Architecture Used
6. Challenges Faced

📊 Evaluation Criteria

Section	Marks
API Integration	20
Visit Feature	10
UI & Map	15
Filtering & Sorting	10
Offline Support	15
Code Quality	5
Documentation	5
Proper AI usage	20

*AI usage will be evaluated by your ai and git usage with code analysis.

🚫 Academic Integrity

- Do NOT copy full projects
- Do NOT share complete solutions
- Code similarity will be checked
- Violations will result in zero marks

You may:

- Discuss ideas
- Share small snippets

💡 Tips

- Start with API integration first
- Then implement UI
- Then add offline support
- Test frequently

⏰ Deadline

- Strict deadline by 15th April 2026 midnight. (no late submission accepted)
- Submission via form: https://forms.gle/PCYop7FHwd8FFbtB8
- Only ONE submission allowed

✅ Final Note

This assignment tests your ability to:

- Work with real APIs
- Handle dynamic data
- Manage offline behavior
- Build complete mobile applications

Additional Resources:

1. Google Maps Android API
   https://developers.google.com/maps/documentation/android-sdk/overview
2. OpenStreetMap Integration in Android
   https://wiki.openstreetmap.org/wiki/Android
3. Retrofit, A Type-Safe HTTP Client for Android
   https://square.github.io/retrofit/
4. Save Data in a Local Database Using Room
   https://developer.android.com/training/data-storage/room
5. CameraX
   https://developer.android.com/media/camera/camerax
6. Photo Picker
   https://developer.android.com/training/data-storage/shared/photo-picker
7. Get the Last Known Location
   https://developer.android.com/develop/sensors-and-location/location/retrieve-current

Good luck with your exam!